

Software Design Specification

Bridging Healthcare Gaps with AI

Project Code:

BHG-AI-(25-26)

Project Advisors:

Dr. Hafiz Muhammad Faisal Shehzad
Mr. Aamir Zia

Project Manager:

Dr. Muhammad Ilyas

Project Team:

Aqsa Shehzad	BSCS51F22S028	(Team Lead)
Farwa Eiman Arif	BSCS51F22S014	(Team Member)
Sehrish Akram	BSCS51F22S019	(Team Member)
Ayesha	BSCS51F21R036	(Team Member)

Submission Date:

December 12, 2025

Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	Bridging Healthcare Gaps with AI
Document	Software Design Specification
Document Version	1.0
Identifier	BHG-AI-(25-26)
Status	Draft
Author(s)	Aqsa Shehzad BSCS51F22S028 (Team Lead) Farwa Eiman Arif BSCS51F22S014 (Team Member) Sehrish Akram BSCS51F22S019 (Team Member) Ayesha BSCS51F21R036 (Team Member)
Approver(s)	PM
Issue Date	Sept. 15, 2025
Document Location	
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
API	Application Programming Interface
DD	Design Specification
PM	Project Manager
AI	Artificial Intelligence
API	Application Programming Interface
FCM	Firebase Cloud Messaging
CRUD	Create, Read, Update, Delete
E/R	Entity - Relationship
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MVC	Model - View - Controller
NLP	Natural Language Processing
ML	Machine Learning
ORM	Object Relational Mapper
OOP	Object - Oriented Programming
PMC	Pakistan Medical Commission
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDK	Software Development Kit
SOLID	Single Responsibility, Open - Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
SQL	Structured Query Language
UML	Unified Modeling Language
URI	Uniform Resource Identifier
WebRTC	Web Real - Time Communication

Table of Contents

1. Introduction	4
1.1 Purpose of Document	4
1.2 Project Overview	4
1.3 Scope	4
2. Design Considerations	6
2.1 Assumptions and Dependencies	8
2.2 Risks and Volatile Areas	12
3. System Architecture	10
3.1 System Level Architecture	10
3.2 Sub-System / Component / Module Level Architecture	11
3.3 Sub-Component / Sub-Module Level Architecture (1...n)	12
4. Design Strategies	19
4.1 Strategy 1...n	19
5. Detailed System Design	14
6. References	23

1. Introduction

This document explains the overall design of SYNC and how different parts of the system work together. It is mainly prepared for developers, testers, project manager, and advisors so they can understand the project structure and workflow easily.

1.1 Purpose of Document

This document describes the architecture and design decisions behind SYNC. It is meant to serve as a reference for developers building the system, testers writing test cases, the project manager tracking progress, and advisors evaluating the technical approach. The design follows Object-Oriented principles and uses a layered architecture to keep the presentation, business logic, and database layers cleanly separated from each other.

1.1.1 Intended Audience

This document is intended for the following stakeholders:

1. Development Team
2. Project Managers
3. Advisors and Evaluators

1.1.2 Design Methodology

The project follows Object-Oriented Design (OOD). The system is divided into separate parts so the code stays cleaner, easier to manage, and simpler to update later.

Main approach used:

- Layered architecture
- Modular design
- API-based communication
- Reusable components
- Simple service-based structure

The system also follows:

- SOLID principles
- REST API design
- Secure authentication
- Error handling

1.2 Project Overview

SYNC is a healthcare web platform made for patients and doctors in Pakistan. It lets patients talk to an AI health assistant, find and book doctors, manage their appointments, track prescriptions, and set medication reminders, all in one place. The AI agent is built on a RAG pipeline trained on sources like MedlinePlus, CDC, WHO, NHS, and Mayo Clinic, and it remembers past conversations so it can give personalized responses over time.

1.2.1 Core Functionality

The system provides the following key capabilities:

1. AI Health Assistant (Primary Feature)

It takes natural language health queries, pulls relevant information from the medical knowledge base in Qdrant, and responds using the Kimi K2 LLM through the Groq API. It classifies urgency into four levels (Non-urgent, Moderate, Urgent, Emergency), suggests the right specialist, and keeps conversation history in memory. A guardrails layer checks every response before it reaches the patient to catch anything unsafe or out-of-scope.

2. Doctor Consultation System

It has a searchable doctor directory with profiles showing specialty, city, fees, and ratings. Patients can book appointments by choosing a date, time, and consultation type. Online appointments get a Whereby video room link automatically. Appointment statuses go through pending, confirmed, completed, and cancelled.

3. Prescriptions and Visit History

After consultation, doctors can add prescriptions and visit notes. Patients can later check their previous medical records from the dashboard.

1.2.2 Design Approach (Tech Stack)

Frontend	React 19, TypeScript, Vite 7, Tailwind CSS, and shadcn/ui
Backend	Python and FastAPI
Database	PostgreSQL via Supabase
Authentication	Supabase Auth
AI Agent	LangChain with Groq
Vector Storage	Qdrant
Embeddings	Voyage AI
Video Calls	Whereby SDK

1.2.3 Key Design Principles

- 1. Modular Code:** Independent, loosely-coupled modules for maintainability
- 2. Scalability:** Horizontal scaling capability for handling increased load
- 3. Secure System:** End-to-end encryption, secure authentication, data privacy
- 4. Reliability:** Fault tolerance, error handling, graceful degradation
- 5. Performance:** Caching, lazy loading, optimized database queries
- 6. Maintainability:** Clean code, comprehensive documentation, design patterns
- 7. Usability:** Intuitive UI/UX, accessibility standards, responsiveness across devices

1.3 Scope

The scope section defines what features are included, excluded, and the operational boundaries of the system.

1.3.1 In-Scope Deliverables:

- AI Health Assistant with RAG, guardrails, memory, and urgency triage.
- Doctor search.
- Appointment booking and management.
- Video consultations.
- Prescriptions and visit history tracking.
- Medication reminders with notifications.
- English Language Support.

1.3.2 Out-of-scope Exclusions:

- Real medical diagnosis.
- Medical imaging.
- Wearable device support.
- Insurance integration.
- Family accounts.

1.3.3 System Boundaries

The system is made for Pakistan and follows PMC-based doctor verification.

2. Design Considerations

This section outlines design-specific assumptions and dependencies that guide architectural decisions.

2.1 Assumptions and Dependencies

Design-level assumptions and dependencies establish the foundation for technology choices and integration strategies.

2.1.1 Design-Level Assumptions

These assumptions were considered during system design and development.

1. Technology Assumptions

React and FastAPI will remain supported during development. Supabase and PostgreSQL will stay available and stable. LangChain with the Groq API (Kimi K2 model) powers the AI agent. LangChain handles the tool-calling agent structure, and Groq provides fast LLM inference at no significant cost for this scale.

2. Infrastructure Assumptions

Frontend is on Vercel, backend on cloud server, and Supabase is used for database. These services reduce setup work for a small team. Whereby SDK is assumed to work in modern browsers. It handles video calls and creates room links automatically when booking is done.

3. Data and Content Assumptions

Publicly available content from MedlinePlus, CDC, WHO, NHS, and Mayo Clinic is assumed to be accessible for ingestion into Qdrant. These are trusted sources that form the foundation of the RAG pipeline.

4. User Behavior Assumptions

Users should have at least 3G or WiFi connection. They are also assumed to have phone or computer with camera and microphone.

2.1.2 Design-Level Dependencies

Following dependencies are involved, mainly from external services, AI tools, infrastructure, and legal rules.

External Service Dependencies

The system depends on third-party services working properly during use and development.

1. Video Calling SDK and SMS/Email Notifications

The system uses Whereby SDK for online consultations. It creates room links during booking and manages the video call itself.

Messages are sent through external email or SMS services. If a message fails, it is just logged and the system continues normally.

AI/ML Model Dependencies

The AI part depends on external models and medical sources.

2. Groq API and LangChain

The AI agent uses Groq with LangChain. If Groq is not available, the system shows a simple fallback message instead of breaking.

3. Medical Knowledge Base

Data from WHO, CDC, NHS, Mayo Clinic, and MedlinePlus is stored in Qdrant for RAG. If these sources change, the ingestion process may need updates.

4. Qdrant and Voyage AI

Qdrant stores vectors and Voyage AI creates embeddings. If one fails, the system runs without RAG support.

Infrastructure Dependencies

The system depends on cloud infrastructure to provide required compute, storage, and networking resources.

5. Database, Supabase and Server Capacity

The whole system depends on Supabase being available for both authentication and PostgreSQL hosting. Connection pooling is handled through SQLAlchemy's session factory.

The backend server needs enough resources to handle AI inference calls and concurrent API requests. FastAPI's async handling helps here but there's no auto-scaling configured at this stage.

Legal and Regulatory Dependencies

The project compliance depends on adherence to applicable data protection, privacy, and industry regulations and it also depends on licensing terms of third-party tools remaining unchanged during the project.

6. Doctor Verification

Doctors must provide a valid PMC number. The system checks it through a verification module, and manual review is used if needed.

7. Medical Disclaimer Rules

The system must always show medical disclaimers. The AI also avoids giving doctor-like final diagnoses.

2.2 Risks and Volatile Areas

This section covers the main risks that could affect the system and areas where things might change during or after development.

2.2.1 Technical Risks

We have analyzed the following technical risks:

1. AI Model Accuracy and Reliability

The AI assistant might give inaccurate or misleading health information, which could cause patients to make wrong decisions about their health. This is probably the biggest risk in the whole project.

Mitigation:

- RAG confidence thresholding is applied (0.6 inclusion, 0.7 high-confidence)
- Guardrails module scans every response before it reaches the patient
- Medical disclaimers are shown throughout the interface
- Agent escalates to doctor recommendation when confidence is low

2. Third-Party Service Failures

The system relies heavily on Groq, Qdrant, Voyage AI, Whereby, and Supabase. Any of these going down affects core functionality.

Mitigation:

- RAG pipeline fails open and returns empty result instead of crashing
- Agent executor catches all failures and returns a friendly fallback message
- Supabase and Whereby outages are outside our control but the rest of the UI still works

3. Video Consultation Quality Issues

Pakistan's network conditions are variable, especially in rural areas. Poor connectivity can make Whereby video sessions unusable.

Mitigation:

- Whereby handles adaptive quality internally
- Patients and doctors can fall back to text chat if video does not work

2.2.2 Security and Privacy Risks

One critical security risk addresses data breaches with comprehensive protection strategies.

4. Unauthorized Access to Patient Data

Medical data is sensitive. If authentication is misconfigured or tokens are mishandled, patient records could be exposed.

Mitigation:

- JWT validation enforced on every protected endpoint via FastAPI dependency
- Role-based access control at the service layer
- Passwords hashed with bcrypt
- All data in transit protected by HTTPS

2.2.3 Requirement Change Risks

We found these risks that address evolving requirements for AI features, regulatory compliance, and user interface design.

5. AI Prompt and Model Changes

As the project develops, the prompts and model configuration will likely need tuning based on testing and feedback. This is expected to happen frequently.

Mitigation:

- All prompts are isolated in prompts.py and can be changed without touching agent logic
- Model name and API key are environment variables, so swapping models requires no code change

6. Regulatory or Disclaimer Requirements

Pakistan's telemedicine regulations are still evolving. Disclaimer language or doctor verification requirements might need to change.

Mitigation:

- Disclaimer text is in the frontend and easy to update
- PMC verification logic is isolated in pmc_verifier.py and can be updated independently

3. System Architecture

3.1 System Level Architecture

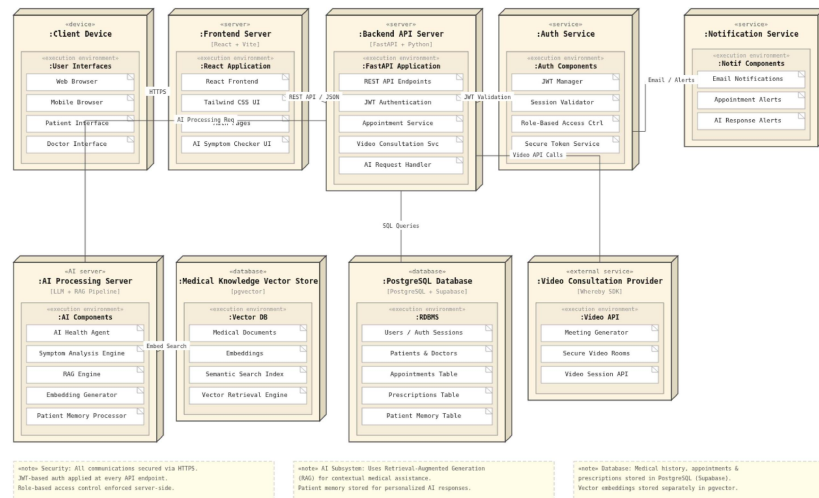
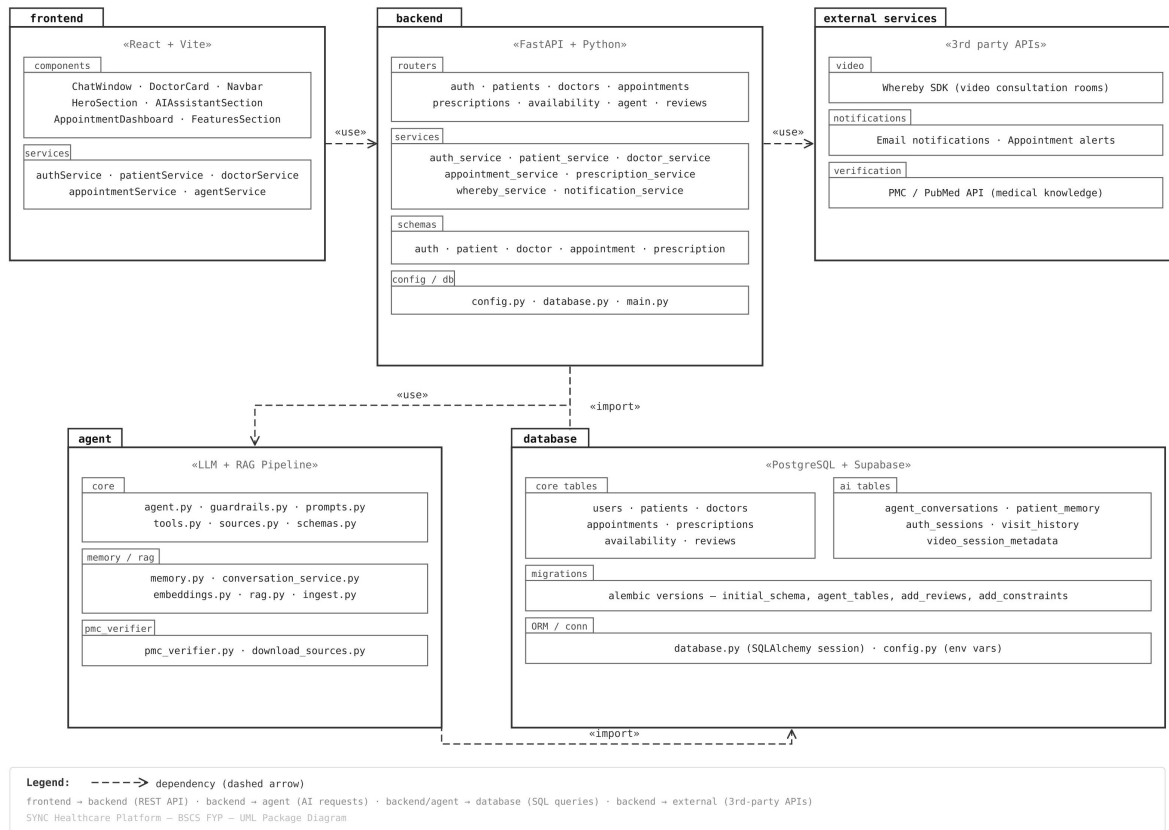
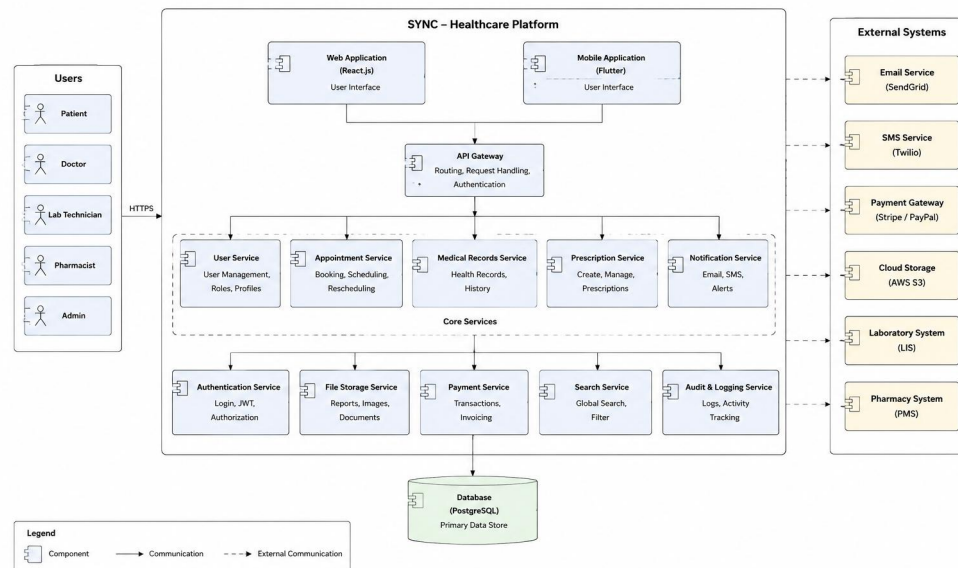


Figure 1 Deployment Diagram

Figure 2 [Package Diagram](#)

3.2 Sub-System / Component / Module Level Architecture

Figure 3 [System Level Component Diagram](#)

3.3 Sub-Component / Sub-Module Level Architecture (1...n)

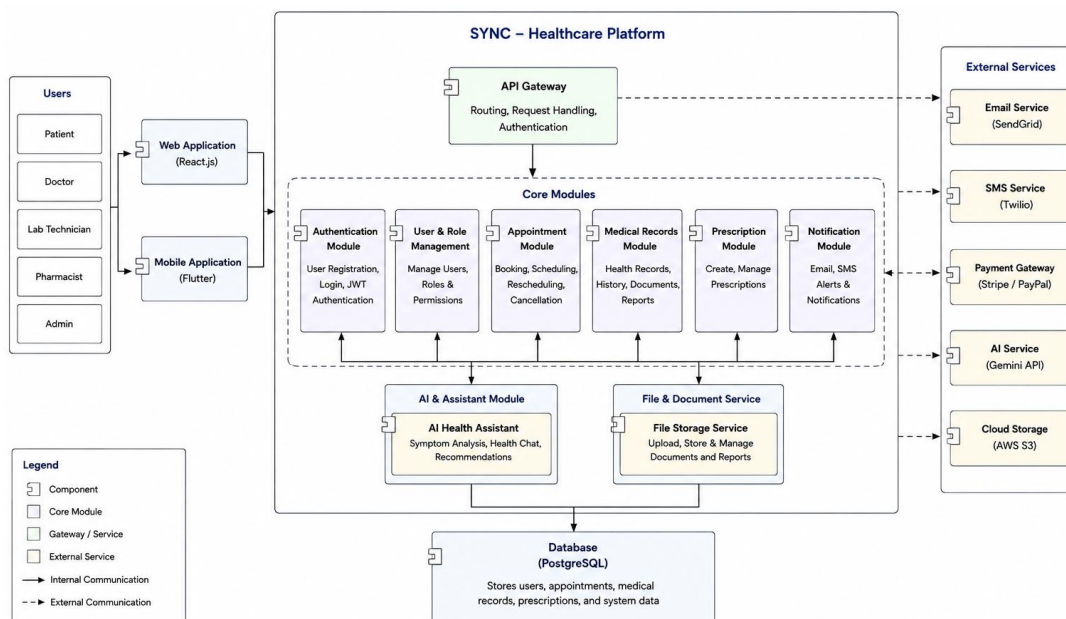


Figure 4 [Sub-System Level Component Diagram](#)

4. Design Strategies

This section explains the main design choices we made in SYNC and the reasons behind them. We focused on keeping things simple, practical, and suitable for both timeline and future scaling.

4.1 Layered Architecture

SYNC is divided into three layers: React frontend, FastAPI backend, and database/AI layer. Each layer has its own role and does not directly mix with others.

This makes development easier because frontend and backend can be worked on separately. If one layer has an issue, it does not affect the whole system. We used Pydantic schemas to manage data passing between layers.

4.2 Modular Backend with a Microservices Mindset

The backend is a single FastAPI app but split into modules like auth, doctors, appointments, prescriptions, reviews, and AI agent. Each module has its own router and service.

We chose a monolith because microservices would be too complex for a small team and short deadline. Still, the structure is modular, so parts like the AI agent can be separated later if needed.

4.3 Domain-Driven Design

Database models are based on real healthcare entities like Patient, Doctor, Appointment, and Prescription. Business logic is placed inside service files instead of routes. This keeps the system closer to real healthcare flow and makes the code easier to understand as features grow.

4.4 Repository-style Data Access

All database access is handled through service classes, not directly inside routes. SQLAlchemy is used for ORM and Alembic for migrations, while Supabase hosts PostgreSQL.

For vector search, Qdrant is used. All access goes through a single abstraction in `rag.py`, so changing the vector database later would be simple.

4.5 AI Agent Architecture

The AI agent uses LangChain with Groq (Kimi K2 model). It decides whether to answer directly or search the medical knowledge base.

The knowledge base is built using data from WHO, CDC, NHS, Mayo Clinic, and MedlinePlus. It is embedded using Voyage AI and stored in Qdrant with cosine similarity search.

If confidence is low, the system shows a warning message. Patient chat history is stored in PostgreSQL so the agent can remember past conversations. Every response also passes through guardrails before being shown.

4.6 Authentication and Security

Authentication is handled by Supabase Auth using JWT tokens. The backend verifies users through a shared dependency used in protected routes.

Patients and doctors are separated using role-based access. Sensitive keys are stored in environment files. CORS is also enabled to restrict API access.

Advanced features like 2FA and audit logs were not added, but the structure supports adding them later.

4.7 Adapter Pattern for External Services

All third-party services are wrapped inside separate files. For example, Whereby is handled in its own service file, and notifications also have a separate module.

This avoids mixing external APIs with core logic. If a provider changes, only one file needs to be updated.

4.8 Responsive Frontend with Component-Based UI

Frontend is built using React, TypeScript, Vite, Tailwind, and shadcn/ui. UI is split into reusable components like DoctorCard, ChatWindow, and Navbar.

Tailwind makes the design responsive across devices. Playwright is used for testing main user flows like login and booking. The component-based structure makes adding new pages easier.

4.9 Testing Strategy

Both backend and frontend are tested. Backend uses Pytest with test cases for auth, appointments, AI agent, RAG, and security. Frontend uses Vitest and Playwright for unit and end-to-end testing. We did not aim for full coverage, only the parts that are critical and most likely to fail or break the system.

5. Detailed System Design

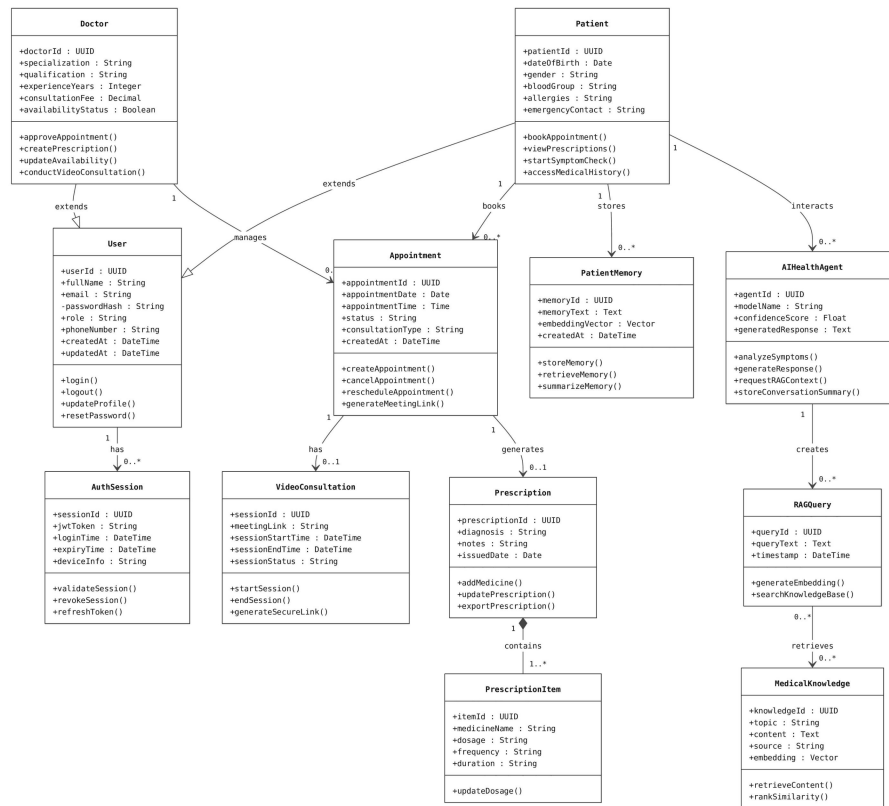


Figure 5 [Class Diagram](#)

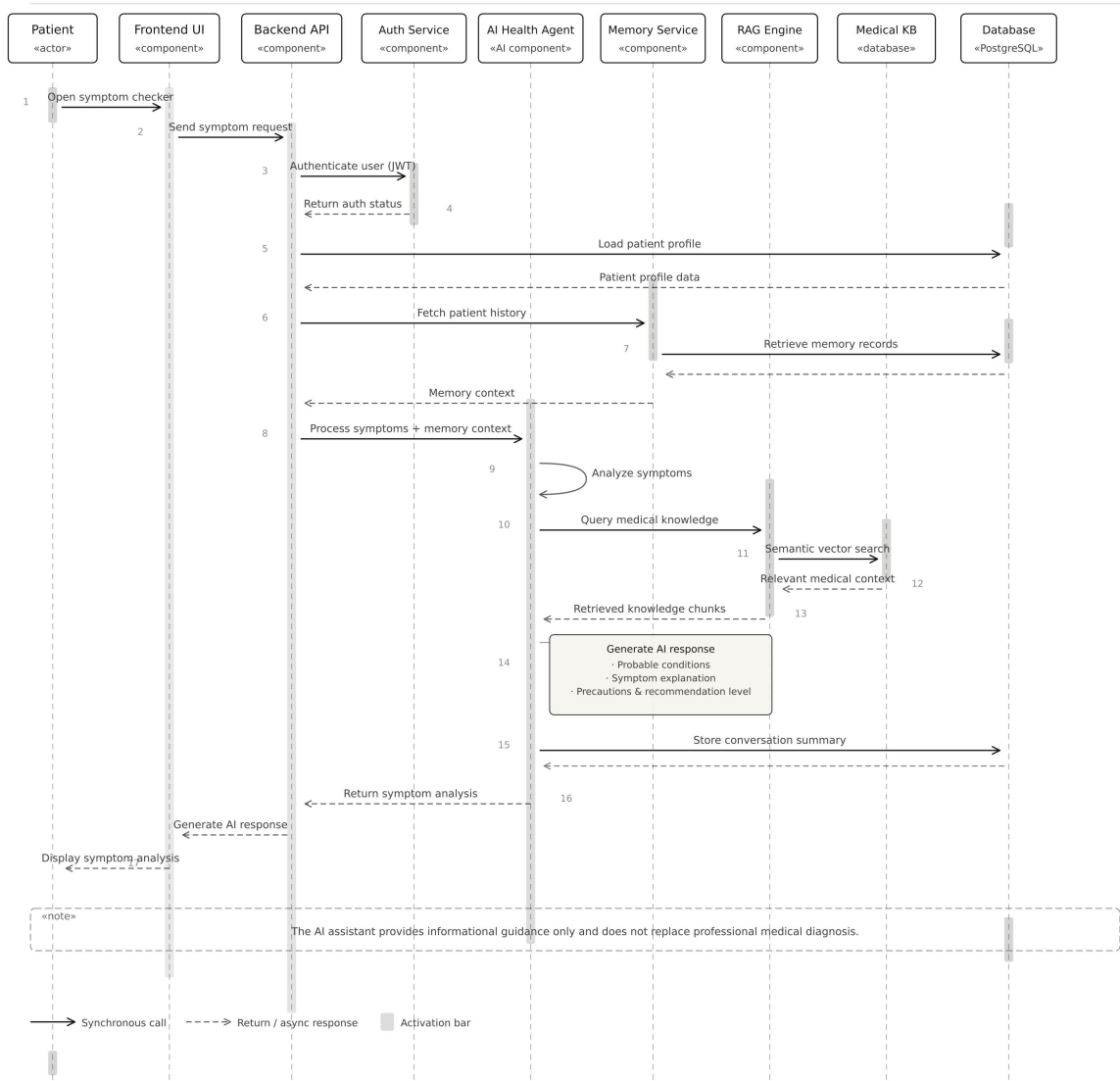


Figure 6 Patient uses AI Health Assistant for symptom Check

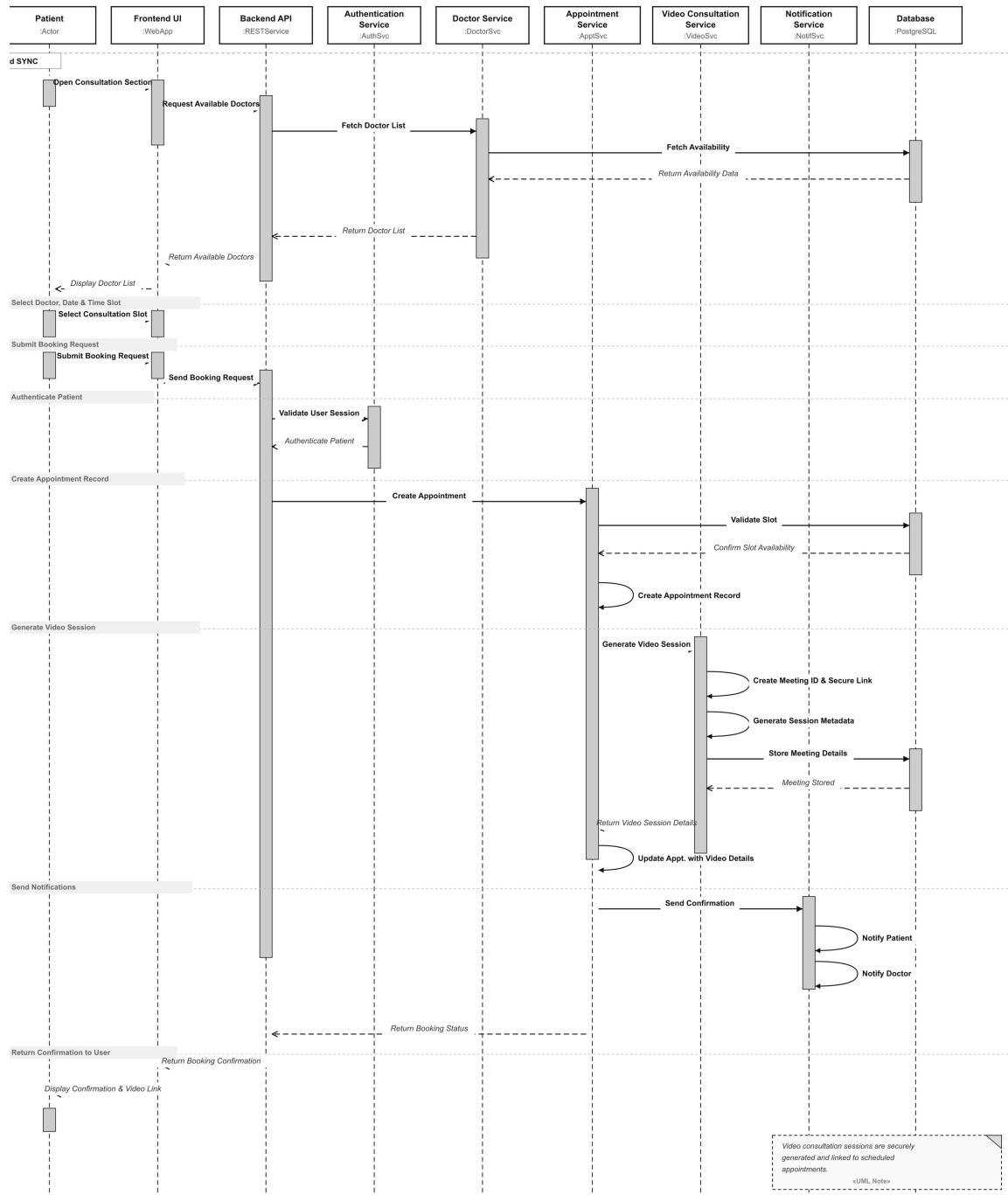


Figure 7 Patient books video consultation with doctor

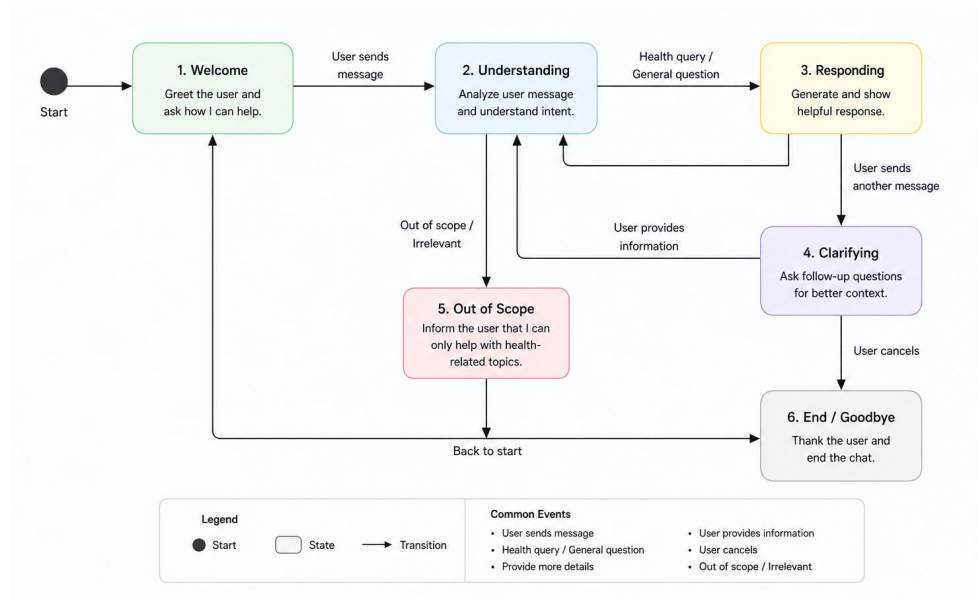


Figure 8 Health Assistant Chat with Patient

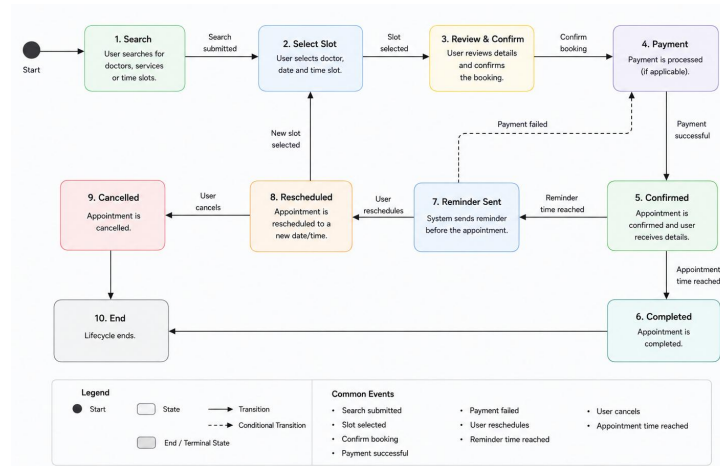


Figure 9 Appointment Lifecycle State Transition Diagram

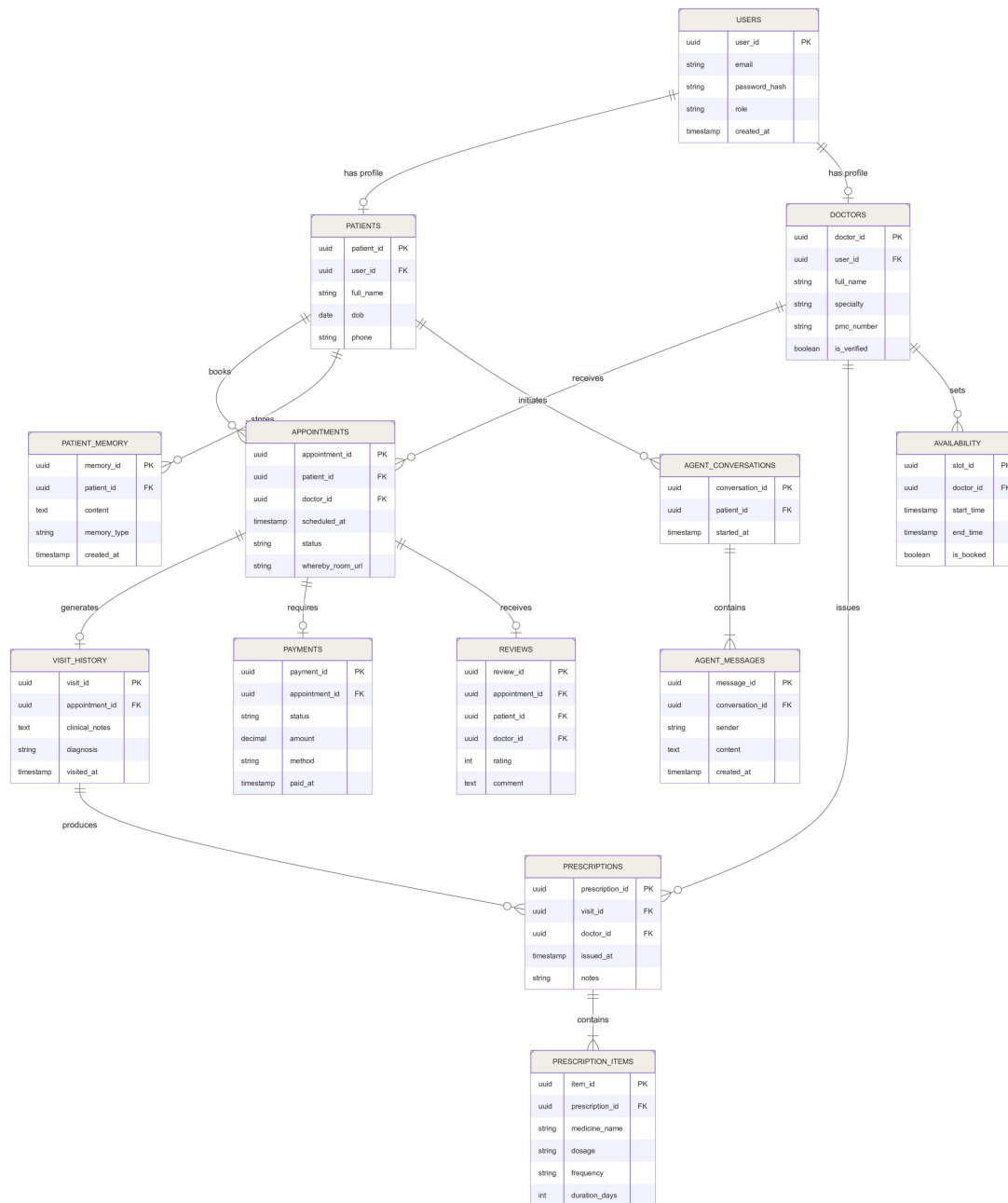
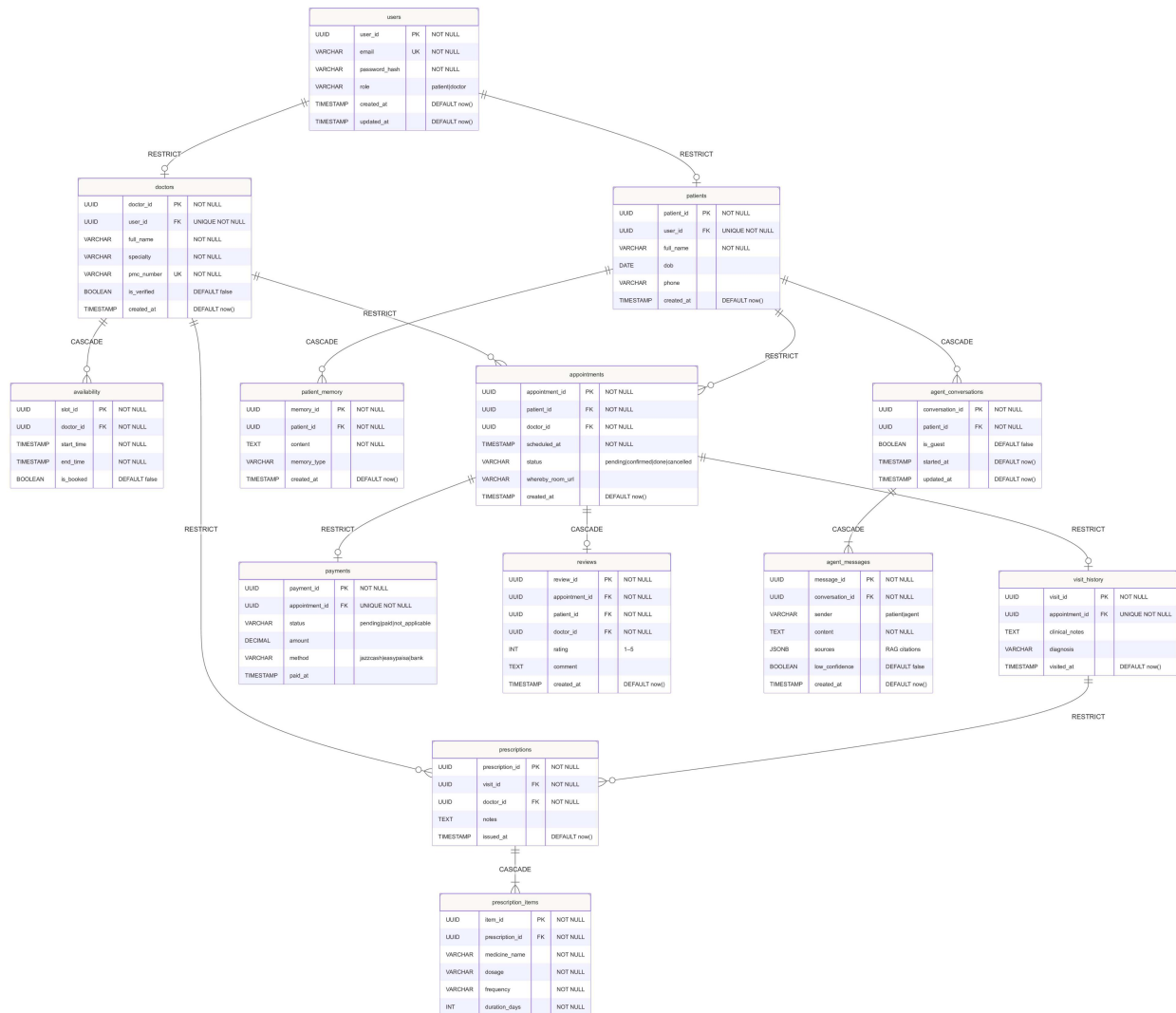


Figure 10 Entity Relationship Diagram (Logical Data Model)

Figure 11 Physical Data Model

Detailed GUIs

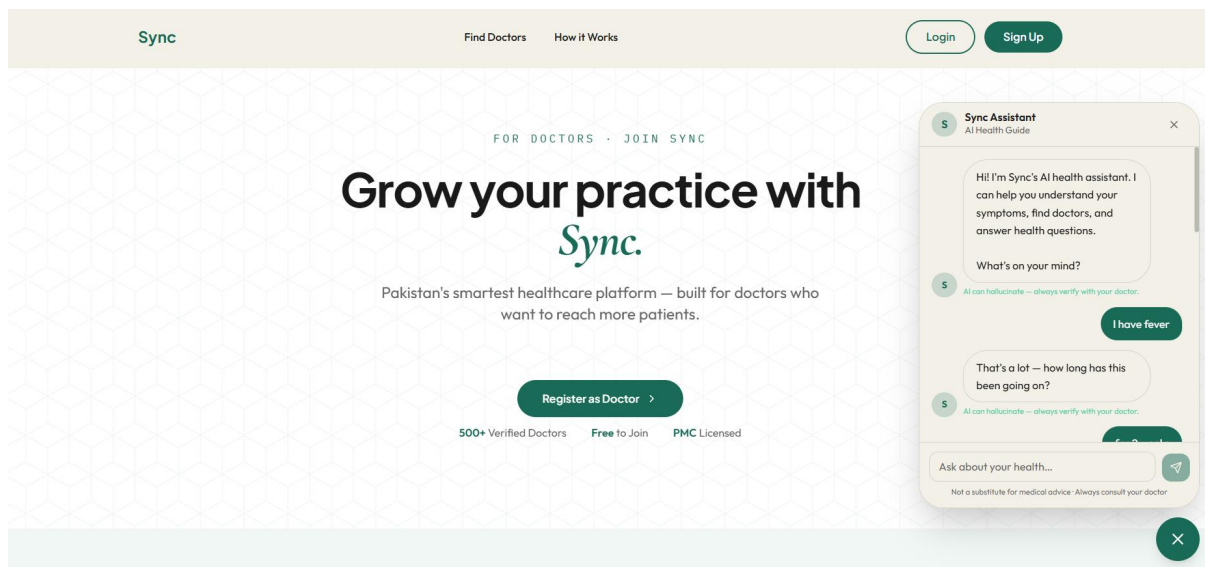


Figure 12: Health Assistant Chat

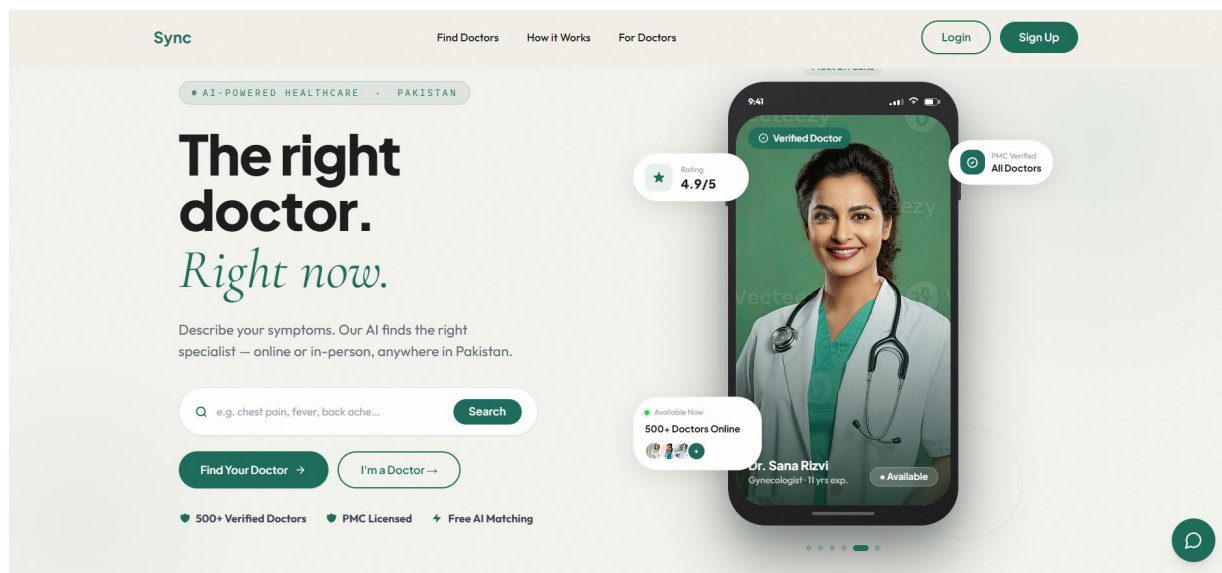
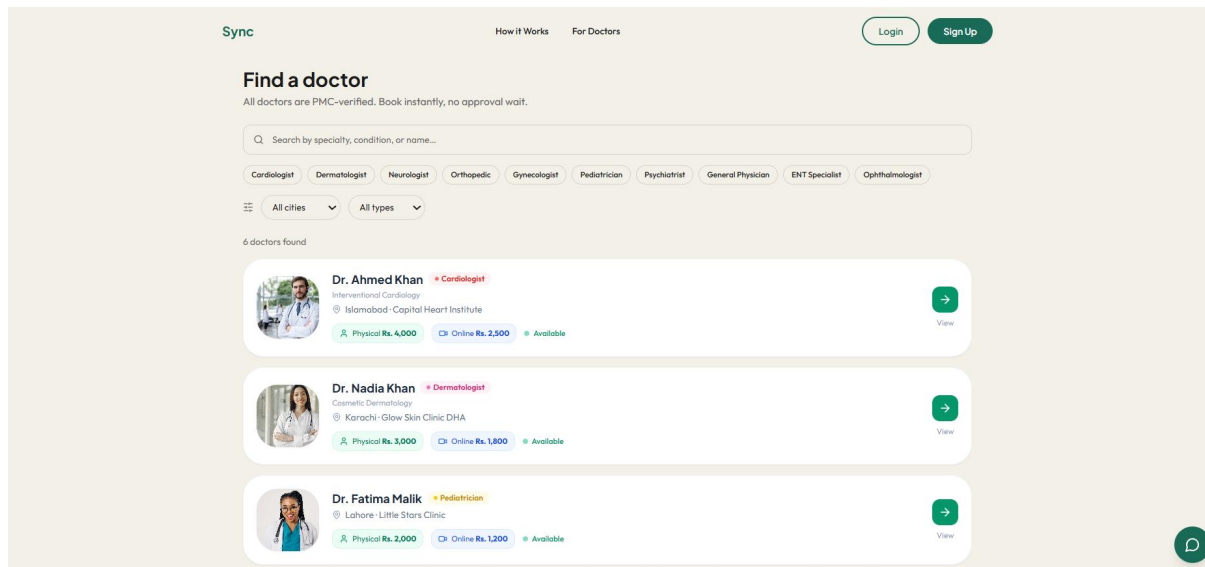
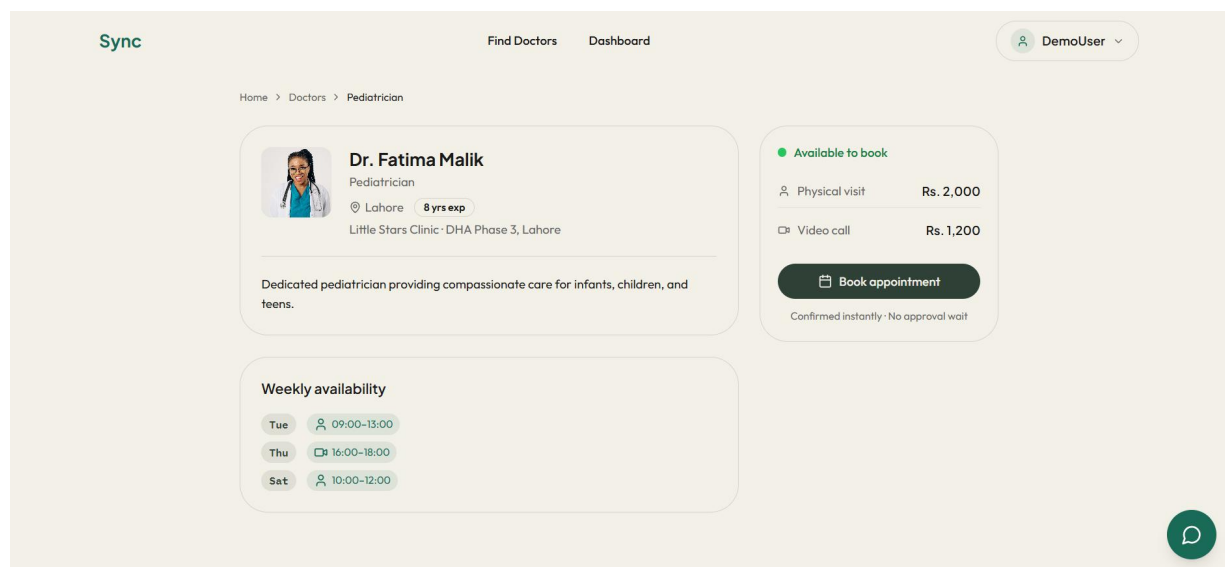


Figure 13: Landing Page

**Figure 14: Doctor Search****Figure 15: Doctor Profile**

The screenshot shows a booking interface for Dr. Fatima Malik, a Pediatrician in Lahore. The page has a header with the 'Sync' logo, 'Find Doctors', 'Dashboard', and a user profile 'DemoUser'. A 'Back to profile' link is at the top left. The main heading is 'Book with Dr. Fatima Malik'. Below it, there are two steps: 1. 'Choose consultation type' with options 'Video call' (Rs. 1,200) and 'Physical visit' (Rs. 2,000). 2. 'Select a date' showing a calendar for May 18-24, 2026, with dates 19, 21, and 23 highlighted. A chat icon is in the bottom right corner.

Sync Find Doctors Dashboard DemoUser

< Back to profile

Book with Dr. Fatima Malik

Pediatrician · Lahore

1 Choose consultation type

Video call Rs. 1,200 Physical visit Rs. 2,000

2 Select a date

18 May – 24 May 2026

MON 18 TUE 19 WED 20 THU 21 FRI 22 SAT 23 SUN 24

Figure 16: Booking Details

The screenshot shows the next steps in the booking process. Step 3 is 'Pick a time' with options 10:00, 10:30, 11:00 (selected), and 11:30. Step 4 is 'Reason for visit' with an 'Optional' field for a brief description. Below this is a summary table for the booking: Doctor (Dr. Fatima Malik), Date (Sat, 23 May), Time (11:00), Type (Physical visit), and Fee (Rs. 2,000). A 'Confirm booking' button is at the bottom, with a note 'Pay at clinic · Confirmed instantly'. A chat icon is in the bottom right corner.

Sync Find Doctors Dashboard DemoUser

2 Select a date

18 May – 24 May 2026

MON 18 TUE 19 WED 20 THU 21 FRI 22 SAT 23 SUN 24

3 Pick a time

10:00 10:30 11:00 11:30

4 Reason for visit Optional

Brief description of your concern...

Doctor	Dr. Fatima Malik
Date	Sat, 23 May
Time	11:00
Type	Physical visit
Fee	Rs. 2,000

Confirm booking

Pay at clinic · Confirmed instantly

Figure 17: Booking Date and Time

6. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PGBH01-2026-Proposal	Project Proposal (Updated)	May 14, 2026	CP-1/Updated Documents/Project Proposal.pdf at master · AqsaShahzad04/CP-1
PGBH01-2026-SRS	Software Requirements Specification (Updated)	May 14, 2026	CP-1/Updated Documents/SRS.pdf at master · AqsaShahzad04/CP-1
PGBH01-2025-[1]	IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)	1998	https://ieeexplore.ieee.org/document/720574
PGBH01-2025-[3]	Oladoc – Book Doctors & Healthcare Professionals Online	2025	https://oladoc.com/
PGBH01-2025-[4]	Marham: Healthcare Services in Pakistan	2025	https://marham.pk/
PGBH01-2025-[5]	Sehat Kahani – Telehealth in Pakistan	2025	https://sehatkahani.com/
PGBH01-2026-[8]	Documentations Folder (GitHub)	2026	CP-1/Updated Documents at master · AqsaShahzad04/CP-1
PGBH01-2026-[9]	CP-2 Code Directory (GitHub)	2026	AqsaShahzad04/SYNC